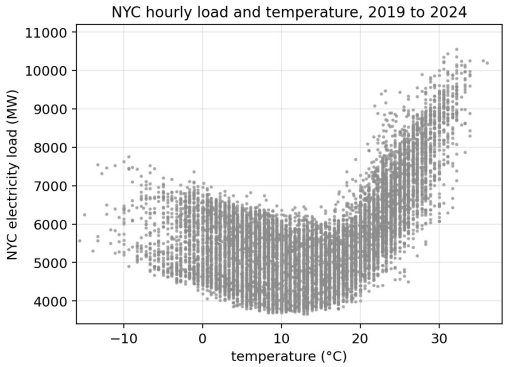
Overview: unsupervised learning and clustering

- **Supervised learning needs a target.** Every sample comes with the answer, and the model learns to reproduce it
- **Unsupervised learning has no target.** The method only sees the inputs. It has to find the structure on its own
- **This week covers three tasks:** clustering (grouping similar samples), density estimation (describing where the data are concentrated), and anomaly detection (finding samples in low-density regions)
- **There are no labels to score against.** A clustering is evaluated by comparing it with information the algorithm did not use, such as a map, a calendar, or known physics

Clustering
Example: grouping US grid cells into climate zones from 12 monthly temperatures and 12 monthly precipitation totals.

Density estimation
Example: describing a cloud of points with a few Gaussian distributions, which gives each point a measure of how typical it is.

Anomaly detection
Example: finding hours when New York City electricity demand was unusual for the temperature, without defining unusual in advance.



Example data set used on several slides. Hourly electricity demand in New York City against temperature, 2019 to 2024. The data carry no labels for season, weekday, or holiday.

Applications of clustering in climate science

Climate zones
Köppen drew his zones by hand from temperature and rain thresholds in 1884. A clustering draws them from the data.

Weather regimes
Winter days over the North Atlantic fall into a few recurring pressure patterns. Forecasters use them for the two to four week range.

Storm genesis regions
Clustering the locations where tropical cyclones form identifies the main formation basins.

Air mass and source types
Aerosol or back-trajectory samples cluster into source regions: dust, smoke, sea salt, traffic.

Sensor behavior
Low-cost sensor errors depend on weather conditions. A mixture model can identify these conditions and correct each one separately. Slide 13.

Anomalies
Faulty instruments, extreme days, and unusual samples: points that do not belong to any group.

- The groups come from the data, not from a person drawing lines on a map
- The groups also depend on choices the person made first. Which features, how they are scaled, how many groups

Distance and feature scaling in clustering

- **Every method this week works on distances between samples.** A sample is a row, a feature is a column, and distance is measured across the columns
- **Without scaling, the feature with the largest values dominates the distance.** New York demand is in thousands of megawatts and temperature in tens of degrees, so unscaled distances mostly reflect demand
- **Standardize, or scale to a fixed range.** StandardScaler gives each column mean 0 and spread 1. MinMaxScaler squeezes each column into a range like −1 to 1
- **Log-transform features that span orders of magnitude,** such as income, emissions, or rainfall, so that a doubling counts the same at low and high values
- **For a map, each grid cell is a sample.** Its features are its values through the year. Scale all months together, so January and July stay comparable. Slide 7
- **Clusters can only reflect the features that are included.** In the demo, the coal-heavy group of countries appears only because a coal-share feature was included

```
from sklearn.preprocessing import StandardScaler
X = StandardScaler().fit_transform(feat)

from sklearn.preprocessing import MinMaxScaler
X = MinMaxScaler((-1, 1)).fit_transform(feat)
```

Comparison with decision trees (Week 5)

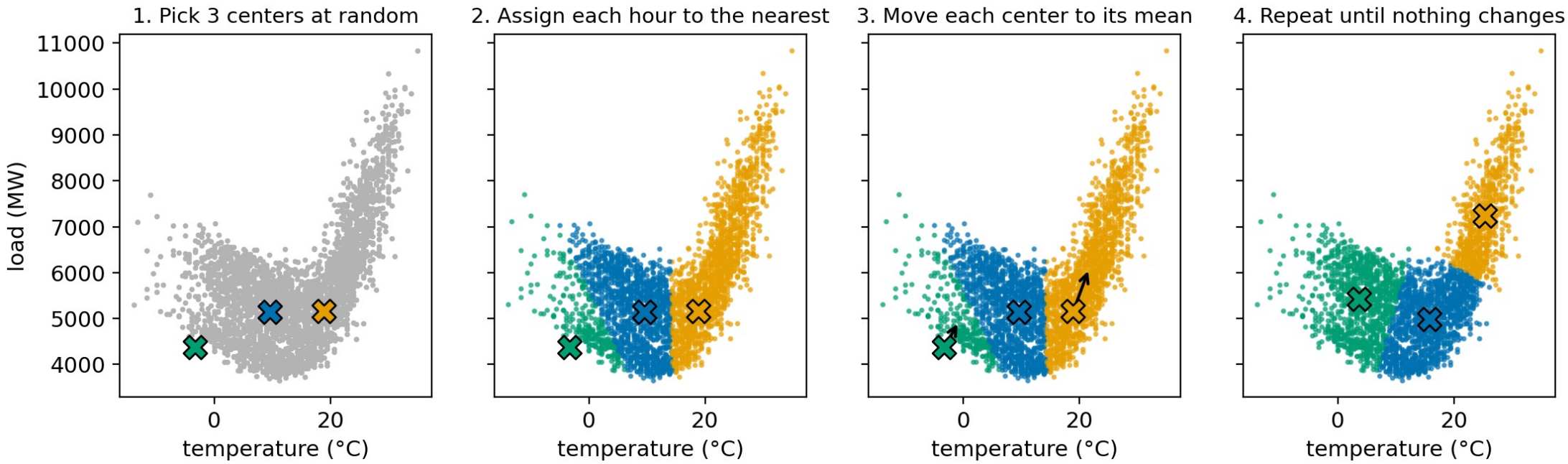
A tree splits on one feature at a time, so feature units do not affect it. A distance combines all features into one number, so their units matter.

Scaling is a modeling choice

Standardizing gives every feature equal weight. That is an assumption, and a feature can be weighted more or less if there is a reason to.

Tutorial, section 1, does the log and standardize steps on the country data. The assignment, Part 2, uses MinMaxScaler on the climate normals

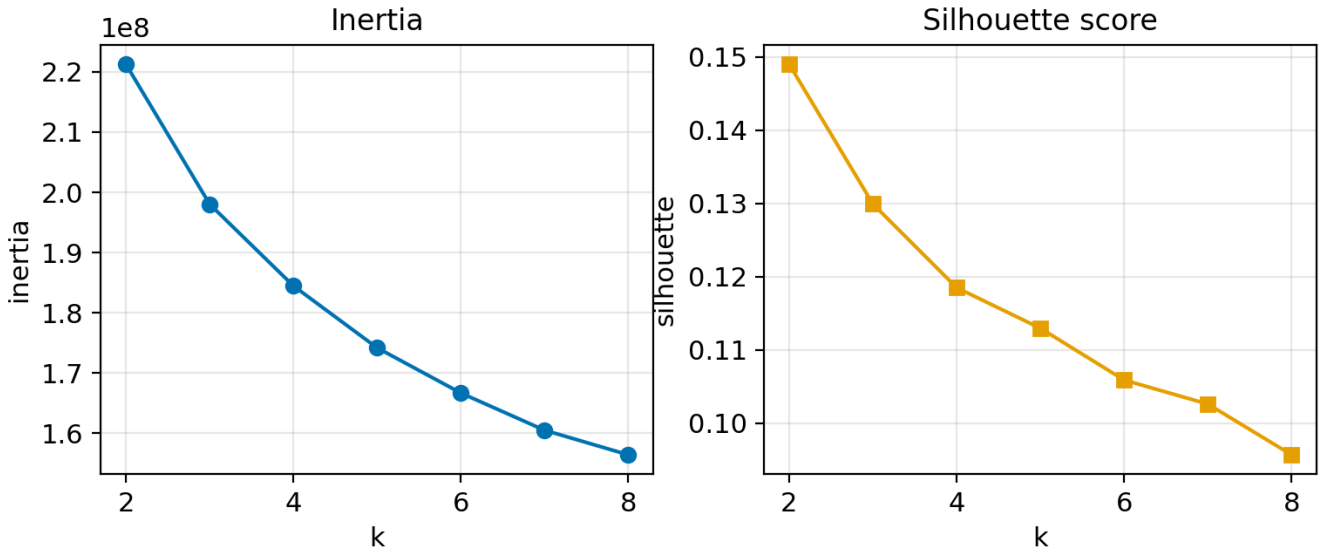
The k-means clustering algorithm



- **Steps:** pick k centers at random, assign each sample to the nearest center, move each center to the mean of its samples, and repeat until the assignments stop changing
- **The result depends on the random starting centers.** scikit-learn runs the algorithm 10 times from different starts and keeps the run with the lowest inertia (n_init=10)
- **Assumptions:** clusters are roughly round and of similar size, and every sample belongs to a cluster. On continuous data like this curve, it divides the data into segments
- **Output:** a label for every sample and k cluster centers. Use inverse_transform to read the centers in original units

3,000 hours of New York demand against temperature, standardized before clustering, plotted in original units. k = 3. Centers are the crosses

Choosing the number of clusters for k-means



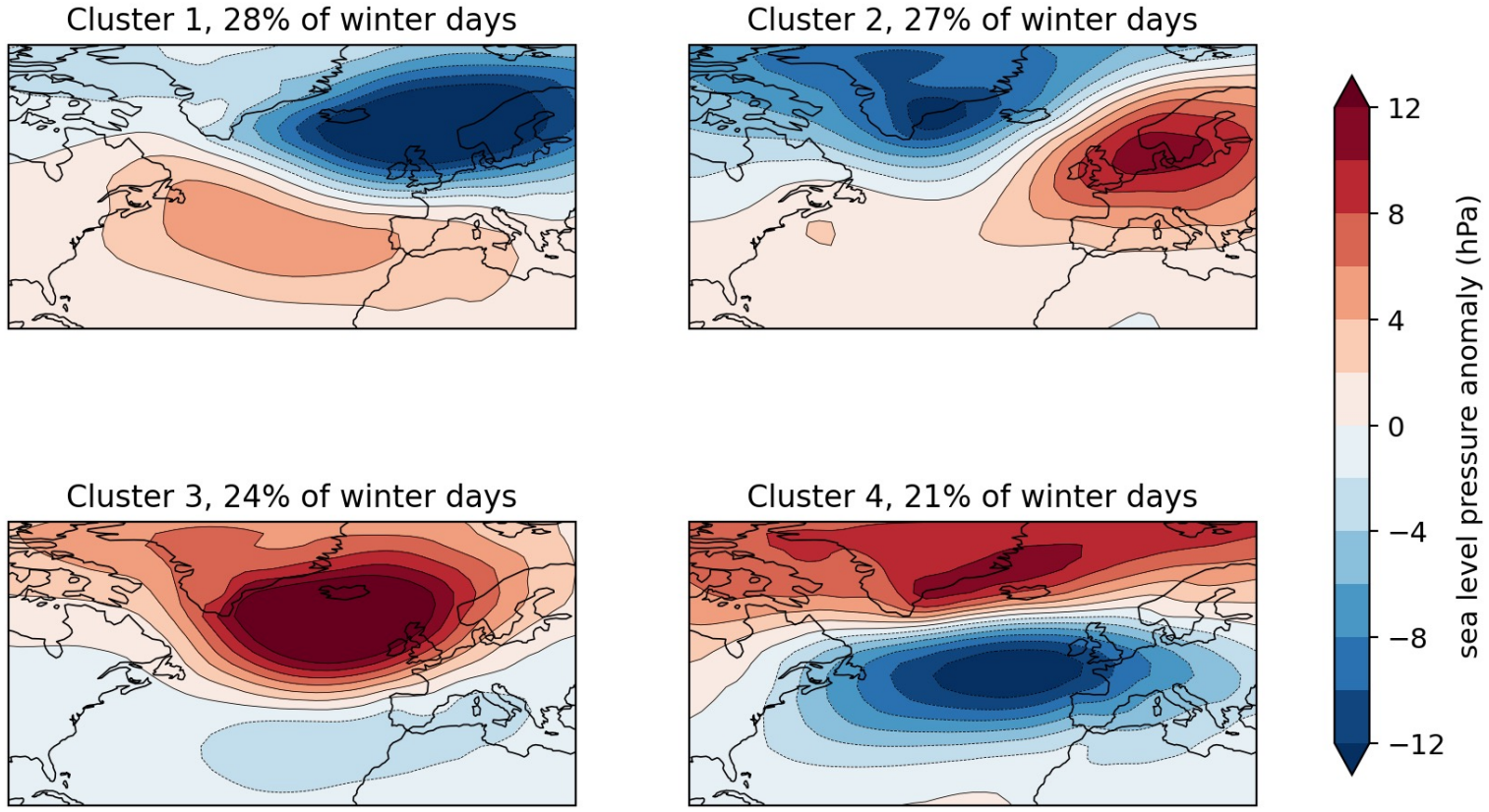
Recommended approach. Use the curves to rule out unreasonable values of k. Then choose k for a stated reason, such as the number of regimes in the literature or the number of zones a map can usefully show. Report the choice and the reason, and check whether the results change for k − 1 and k + 1.

What to avoid. Choosing k only by the highest silhouette over a wide range, and treating that value as the physically correct number of groups.

- **The number of clusters, k, is set by the user.** The algorithm does not choose it
- **Inertia** is the total squared distance from samples to their centers. It always falls as k grows, so look for the bend where it stops falling fast
- **Silhouette** compares each sample's distance to its own cluster with its distance to the nearest other cluster. Near 1 is tight and separate. Near 0 is overlap
- **For the winter pressure maps:** the inertia curve has no clear bend, and the silhouette is at most 0.15. Low values are common in real data, where samples form a continuum rather than separate groups
- **A third check is stability across random starts.** For k = 4, eight single-start runs agree with each other at adjusted Rand index 0.30 to 0.97. Results that change with the random seed should not be interpreted

Winter daily sea level pressure anomalies over the North Atlantic, 1979-2024, 4,152 days, 1225 grid cells reduced to 14 PCs before k-means. Silhouette on a 3,000-day sample

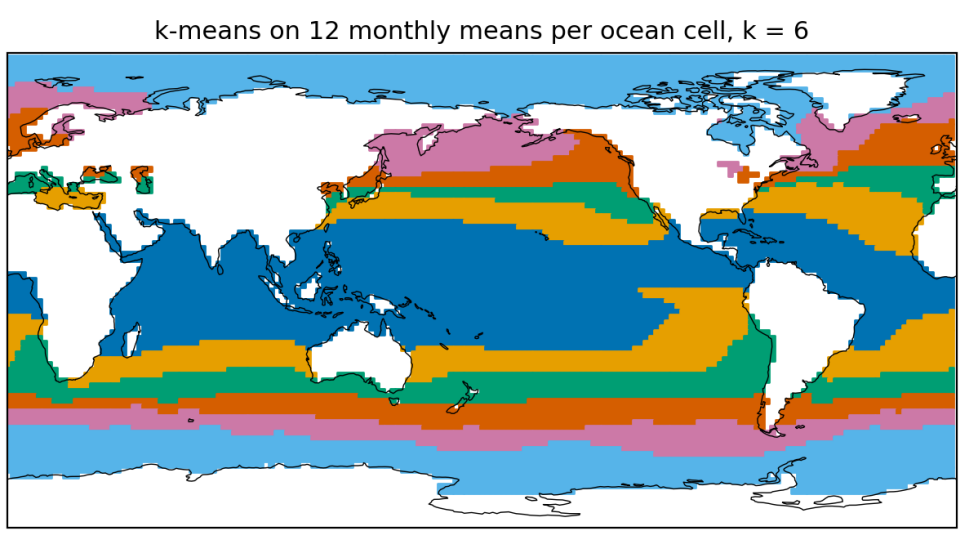
Application: North Atlantic winter weather regimes

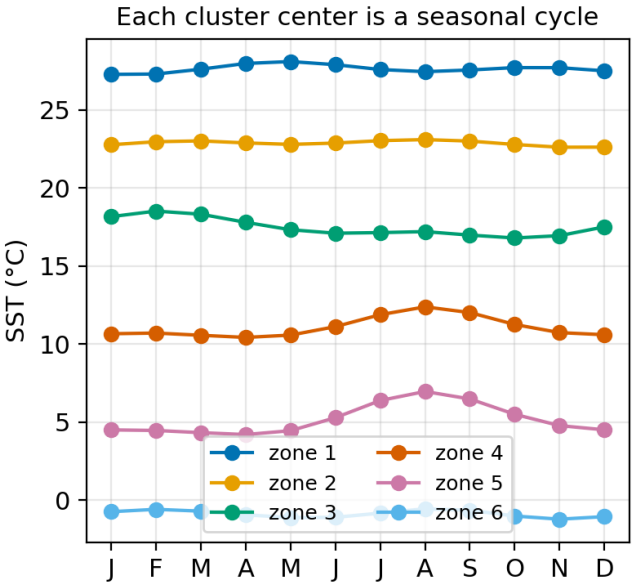


- **The data:** daily sea level pressure, December to February, 1979-2024, from the NCEP reanalysis. One sample is one day's map over the North Atlantic and Europe
- **The steps:** remove the seasonal cycle, weight by area, PCA down to 14 components, k-means with k = 4 and 100 starts
- **Three of the four clusters match regimes described in the literature:** NAO positive (28% of days), Scandinavian blocking (27%), and NAO negative (24%). The fourth, a Greenland high (21%), resembles NAO negative. The Atlantic ridge regime did not appear as a separate cluster here
- **Why regimes are useful:** a regime lasts about 4 days on average and shapes the weather downstream, including European cold spells, wind power output, and storm tracks. Forecast centers issue regime probabilities for weeks two to four
- **Choice of k:** k = 4 follows the literature, and the silhouette does not single it out. Single random starts reach inertias within 0.3% of each other while disagreeing on about a third of the days

Composite anomaly of all days in each cluster, hPa. Method follows Michelangeli, Vautard and Legras 1995. Regime names after Cassou 2008

Clustering gridded data: grid cells as samples





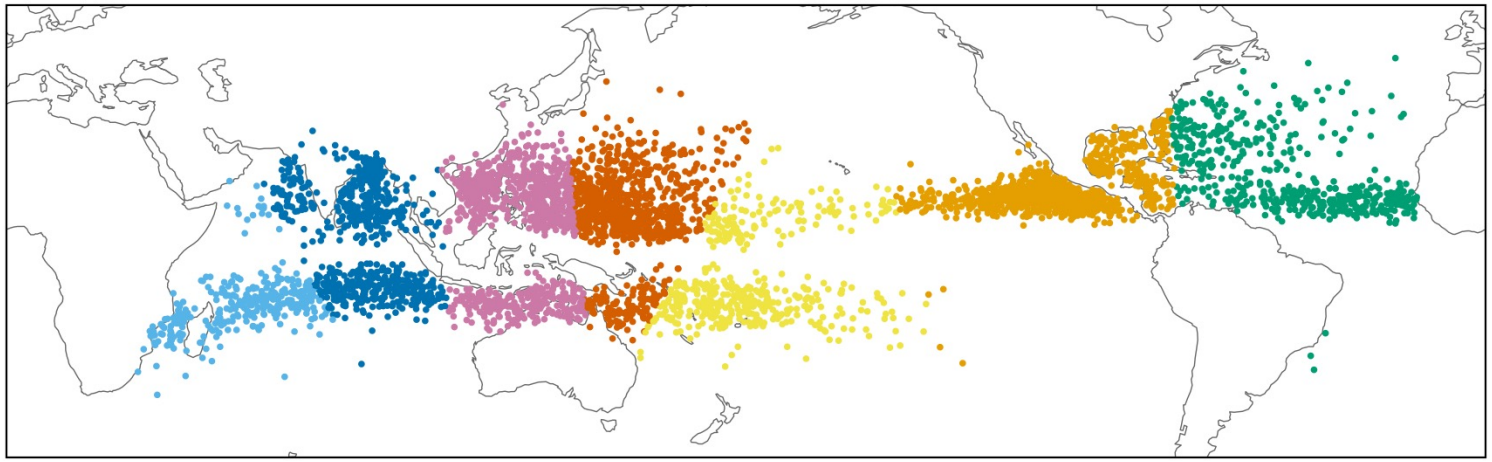

```
vals = clim.values          # 12 x lat x lon
ocean = ~np.isnan(vals[0])  # True where data
feat = vals[:, ocean].T     # samples x 12
lat_m = lat_grid[ocean]
lon_m = lon_grid[ocean]
sc = MinMaxScaler((-1, 1))
X = sc.fit_transform(
    feat.reshape(-1, 1)).reshape(feat.shape)
lab = KMeans(6, n_init=20).fit_predict(X)
plt.scatter(lon_m, lat_m, c=lab, s=4,
            cmap=ListedColormap(colors))
```

- **Each ocean cell is a sample with 12 features, its monthly mean temperatures.** Land cells are dropped with a mask, not filled
- **Scale all 12 months with one MinMaxScaler.** Reshape to one column, scale, and reshape back, so that differences between months are preserved
- **Plot the kept cells as a scatter plot colored by cluster label,** using a discrete colormap with one color per cluster
- **Interpreting the centers:** each center is a seasonal cycle. The zones follow latitude except where ocean currents shift them, for example along the Gulf Stream

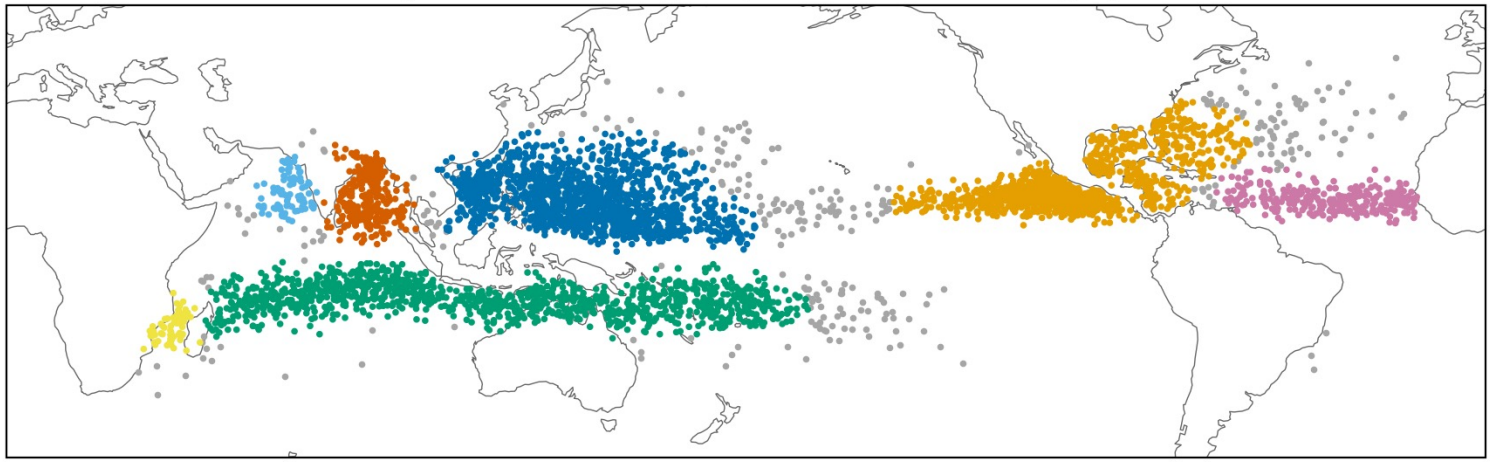
NOAA ERSST v5 monthly climatology 1991 to 2020, 2 degree grid, 10,988 ocean cells, k = 6. The assignment does the same with 12 months of temperature and 12 of precipitation over the United States

DBSCAN: density-based clustering

k-means, k = 7 (every storm is assigned to a cluster)



DBSCAN, eps = 500 km, min_samples = 25: 7 clusters, gray = noise



- **Parameters:** a radius, eps, and a minimum number of neighbors, min_samples. The number of clusters is not specified
- **A core point has at least min_samples neighbors within eps.** Core points within eps of each other, directly or through other core points, form one cluster. Points within eps of a core point that are not core points themselves join its cluster as border points
- **All other points are labeled noise (−1).** They do not belong to any dense region
- **Clusters can have any shape.** A long, curved band forms one cluster if it is dense along its whole length
- **Example: formation points of 4,799 storms since 1980.** k-means with 7 clusters divides the oceans into compact regions with arbitrary boundaries. DBSCAN with a 500 km radius finds 7 dense regions that match the cyclone basins, and labels 310 storms as noise

IBTrACS v4, first recorded position of each storm, 1980 to 2024. DBSCAN uses great-circle distance (metric="haversine"). Gray points in the lower panel are noise

Choosing eps for DBSCAN, and its limitations

eps (km)	clusters	noise points
200	9	3,667
300	18	1,681
400	11	658
500	7	310
700	2	127

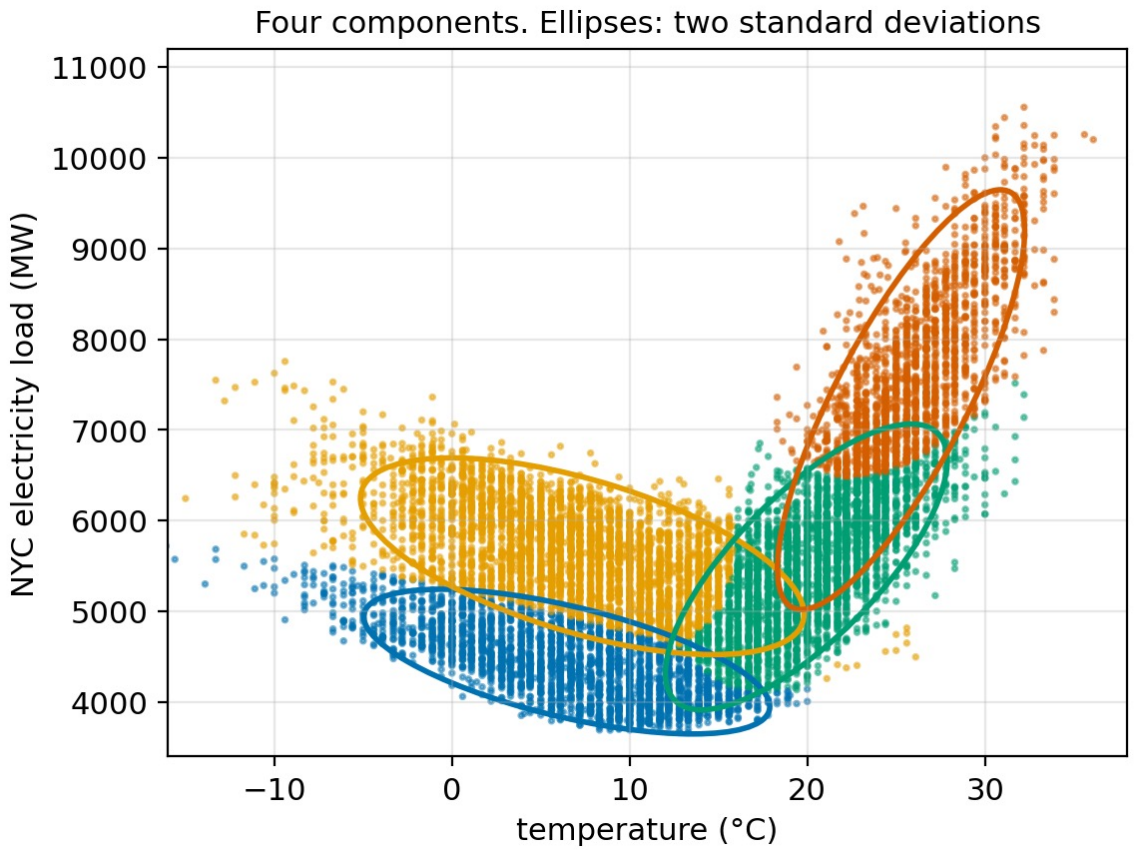
Storms labeled as noise (310): subtropical Atlantic storms that formed near 40°N, storms in the sparse central Pacific, and three South Atlantic storms, including Catarina in 2004.

A limitation visible here: the eastern Pacific and the Gulf of Mexico form one cluster, because their formation regions are within 500 km of each other across Central America and the distance measure ignores land.

- **The result is sensitive to eps.** With a small eps most points are noise, and with a large eps the clusters merge. In between, the number of clusters rises and falls
- **Choosing eps:** a common heuristic is to set min_samples to about twice the number of features, then sort each point's distance to its min_samples-th nearest neighbor and look for the knee in that curve
- **Regions of different density are a limitation.** A single eps cannot suit both a dense basin and a sparse one. Many South Pacific storms are labeled noise because the region is sparse, not because the storms are unusual
- **No predict method.** DBSCAN only labels the data it was fit on. To label a new point, fit a nearest neighbor classifier on the core points, or check whether the point is within eps of a core point
- **Computational cost:** fast with a tree index when eps is small, and slow when eps is large or the data have many features

min_samples = 25 throughout. The haversine metric needs latitude and longitude in radians and eps in radians, so 500 km is 500 / 6371

Gaussian mixture models (GMMs) and soft clustering

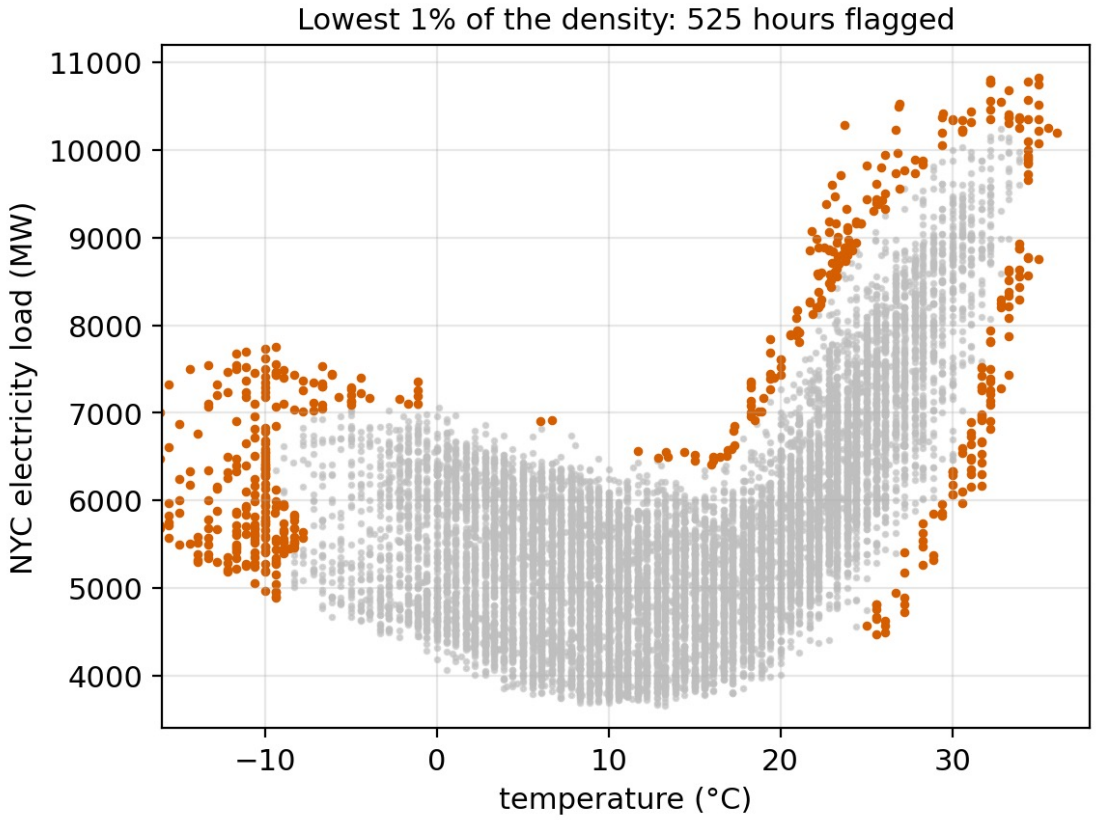


- **Model:** the data are assumed to come from k Gaussian distributions, each with its own mean, covariance, and weight. Fitting the model estimates these
- **Each ellipse is a component.** Its center is the mean and its shape comes from the covariance, so a component can be elongated and tilted. k-means clusters are round and of similar size
- **Membership is a probability.** Each hour gets a probability for each component. For 11% of the hours, no component has a probability above 0.6, because these hours lie where components overlap
- **The fit is by expectation maximization.** Start from initial components, compute each point's membership probabilities (responsibilities), refit the components using those probabilities as weights, and repeat. This is similar to k-means with probabilistic assignments
- **A GMM also gives a probability density.** For any point, the model returns how likely a point like it is under the fitted distribution. The next slide uses this for anomaly detection

```
gm = GaussianMixture(n_components=4, n_init=5).fit(X)
labels = gm.predict(X);  probs = gm.predict_proba(X)
```

Hourly New York City demand and temperature, 2019 to 2024. Four components, full covariance. Ellipses are two standard deviations. Colors are the most probable component

Anomaly detection with a Gaussian mixture model

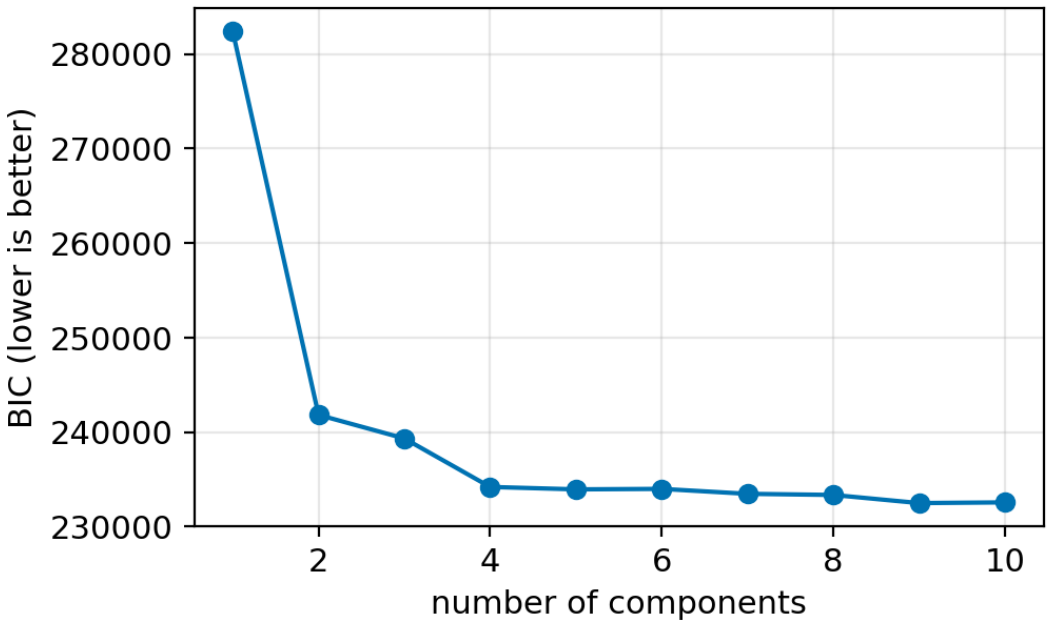


- **score_samples gives the log density at every point.** Points with the lowest density lie in regions with very few training samples
- **Threshold: the lowest 1% of densities, 525 hours.** The threshold is a choice, like k, and can be set by how many flagged samples can be inspected
- **Flagged hours include** the polar vortex outbreak of late January 2019, the December 2022 cold snap, and the July 2019 heat wave: the hours at the extreme ends of the distribution
- **Not flagged:** the spring 2020 lockdown, when demand was well below normal for the temperature. Only 4 of those 1,340 hours were flagged, because they lie inside the cloud near its lower edge
- **Implication:** whether a sample is anomalous depends on the features. The model only flags unusual combinations of the features it was given. Detecting the lockdown would need day of week and hour

```
dens = gm.score_samples(X)
flag = dens < np.percentile(dens, 1)
```

Flagged hours by day: 2019-01-21 (24), 2019-01-31 (24), 2022-12-24 (23), 2019-02-01 (21), 2019-01-22 (19), 2023-02-04 (12). Dates in local time

Choosing the number of components with BIC

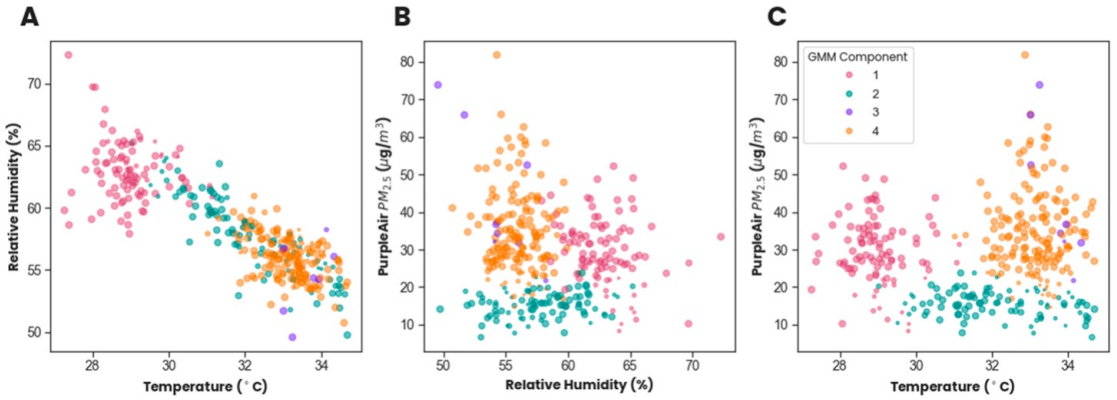


Bayesian Gaussian mixtures. BayesianGaussianMixture takes an upper limit on the number of components and gives near-zero weight to the components it does not need. It still requires a prior, which plays a role similar to the BIC penalty.

- **A mixture model is a probability model, so it has a likelihood.** Adding components always increases the likelihood, so a criterion penalizes the extra parameters
- **BIC = parameters × log(n) − 2 × log likelihood.** Lower is better. AIC uses a smaller penalty, so it tends to select more components
- **For the New York data, BIC decreases up to 9 components.** Most of the decrease happens by 4. Beyond that, each added component improves the fit slightly, because more Gaussians can always approximate a curved distribution a little better
- **We use 4 components,** where the BIC curve flattens and where each component has a clear interpretation. The minimum at 9 reflects the curved shape of the data rather than 9 distinct groups
- **In the tutorial, BIC selects 2 components for the country data.** The tutorial uses 4 for interpretability, and states that choice

BIC for 1 to 10 components on the standardized New York data, 52,439 hours, n_init = 3

Application: calibrating low-cost PM2.5 sensors



Reading the figure. Each daily average is colored by its most probable component. Panel A, humidity against temperature: the components line up along the season. Panels B and C: component 2 (teal) is the low-pollution background, component 4 (orange) is the dry season with high readings. Point size is the membership probability. Small points are the days the model was unsure about.

Summary. Because a mixture is a full probability model of the data, one four-component fit provides clustering, regression, and uncertainty estimates. A random forest provides predictions only.

- **Setting:** a 250 dollar PurpleAir sensor next to a reference monitor at the US embassy in Accra, Ghana, March 2020 to March 2021. Raw agreement $R^2 = 0.53$. The sensor's error changes with the season: it reads high in the humid rainy season and low on some dusty dry-season days
- **The method:** fit a Gaussian mixture to the joint data: sensor PM2.5, temperature, humidity, and reference PM2.5 together. BIC picked 4 components. Then predict the reference value as the mixture's conditional mean given the three inputs. That is Gaussian mixture regression
- **The result:** $R^2 = 0.88$ and mean error 2.2 µg/m³, against 0.81 for linear regression and 0.81 for a random forest on the same inputs
- **Interpretation of the components:** they correspond to regimes of sensor behavior. Component 1 is the rainy season, with high humidity and a sensor that reads high. Component 4 is the hot, dry Harmattan season, with larger errors. A single linear correction cannot represent both
- **Additional advantages:** the model still produces estimates when an input is missing, and it gives an uncertainty for every corrected value. The 95% intervals contained the reference value 96% of the time

McFarlane, Raheja, Malings, Appoh, Hughes and Westervelt 2021, ACS Earth and Space Chemistry 5, 2268. Figure 5, open access

Comparing k-means, DBSCAN, and GMMs

	k-means	DBSCAN	Gaussian mixture
What you choose	k, the number of clusters	eps and min_samples, a density	k, the number of components, and the covariance shape
Cluster shape	Round, similar size	Any shape, if dense throughout	Ellipses of any tilt and size
Outliers	Forced into a cluster	Labeled −1, noise	Low density, you set the threshold
Membership	Hard, one label	Hard, one label or noise	Soft, a probability per component
New samples	predict	None. Use nearest neighbors on the cores	predict and predict_proba
Choosing the size	Elbow, silhouette, stability	Knee in the k-distance plot	BIC or AIC
Scales to	Millions of samples, MiniBatchKMeans	Large n if eps is small	Thousands to a few million, depends on covariance type
Use it for	A quick partition, compressing a field, a first look	Spatial data, odd shapes, finding noise	Overlapping groups, densities, anomalies, uncertainty

- **All three need scaled features, and all three depend on their parameters.** None of them indicates whether the groups are physically meaningful
- **The tutorial runs all three on the same countries and compares the labelings.** Groups found by methods with different assumptions are more likely to be real. A group found by only one method may reflect that method's assumptions
- **Other names you will meet:** hierarchical clustering (a tree of merges, no k until you cut it), spectral clustering (k-means on a graph), HDBSCAN (DBSCAN across densities), self-organizing maps (a grid of k-means centers, popular in oceanography)

Checks before interpreting a clustering

Feature scaling
The scaling determines the distances, and so the clusters. Standardize, log-transform features with large ranges, scale a map's months together, and report what you did.

Number of clusters
Use the elbow, silhouette, or BIC to rule out poor values. Choose k for a stated reason, and check k − 1 and k + 1.

Sensitivity to random starts
Use n_init above 1 for k-means and mixtures. If the labels change with random_state, the result is not robust.

Autocorrelated samples
Consecutive days or hours are not independent. Clusters of autocorrelated samples can look better defined than they are.

Comparison with independent information
Compare the clusters with information the algorithm did not use: a map, a calendar, known physics, or a different method.

Boundaries in continuous data
When the data form a continuum, k-means still divides them into k groups. The boundaries then reflect the choice of k as much as the data.

- **Applying this to the assignment:** when you compare your k-means and Gaussian mixture maps with each other and with the climate zone map, consider the scaling, the choice of k, and the random seed

Extra

Two reference slides. The math behind the three methods, and the scikit-learn calls in one place. Not covered in class.

Equations for k-means, Gaussian mixtures, and BIC

k-means minimizes the inertia, the within-cluster sum of squares

$$J = \sum_{i=1}^{n} \left\| x_i - \mu_{c(i)} \right\|^2$$

c(i) is the cluster of sample i and μ its center. Each step of the algorithm lowers J or leaves it alone: assigning to the nearest center cannot raise it, and the mean is the point that minimizes the squared distances. So it converges, to a local minimum.

A Gaussian mixture is a weighted sum of bell curves

$$p(x) = \sum_{k=1}^{K} \pi_k \, \mathrm{N}(x \mid \mu_k, \Sigma_k)$$

π_k are the shares and sum to 1. μ_k is a center and Σ_k a covariance matrix, which gives the ellipse its size and tilt. score_samples returns log p(x). The likelihood of the data is the product of p(x) over the samples.

BIC charges log n per parameter

$$\mathrm{BIC} = m \log n \; - \; 2 \log \hat{L}$$

m parameters, n samples, L̂ the maximized likelihood. Lower is better. AIC replaces log n with 2.

Expectation maximization, the E step: responsibilities

$$\gamma_{ik} = \frac{\pi_k \, \mathrm{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_j \, \pi_j \, \mathrm{N}\left(x_i \mid \mu_j, \Sigma_j\right)}$$

γ_ik is the probability that sample i came from component k, given the current components. predict_proba returns it.

The M step: refit each component with the responsibilities as weights

$$N_k = \sum_i \gamma_{ik}\,, \qquad \pi_k = \frac{N_k}{n}, \qquad \mu_k = \frac{1}{N_k} \sum_i \gamma_{ik} \, x_i$$

$$\Sigma_k = \frac{1}{N_k} \sum_i \gamma_{ik} \, (x_i - \mu_k)(x_i - \mu_k)^{\mathrm{T}}$$

Alternate E and M until the log likelihood stops rising. Each round raises it or leaves it alone, so it converges to a local maximum. With γ_ik replaced by 0 or 1 for the nearest center, this is k-means.

Silhouette of one sample

$$s = \frac{b - a}{\max(a, b)}$$

a is the mean distance to the sample's own cluster, b the mean distance to the nearest other cluster.

Summary of clustering functions in scikit-learn

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.cluster import KMeans, DBSCAN, MiniBatchKMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score, adjusted_rand_score

X = StandardScaler().fit_transform(feat)      # or MinMaxScaler((-1, 1))

km = KMeans(n_clusters=4, n_init=10, random_state=0).fit(X)
km.labels_                  # cluster of each training sample
km.cluster_centers_         # k by features, in scaled units
km.inertia_                 # the quantity it minimized
km.predict(X_new)           # nearest center for new samples
scaler.inverse_transform(km.cluster_centers_)   # read them

silhouette_score(X, km.labels_)                 # -1 to 1
adjusted_rand_score(labels_a, labels_b)         # agreement of two
labelings

db = DBSCAN(eps=0.6, min_samples=6).fit(X)
db.labels_                  # -1 is noise
db.core_sample_indices_     # which samples are cores
# no predict: fit KNeighborsClassifier on the cores if you need one

gm = GaussianMixture(n_components=4, n_init=5, random_state=0).fit(X)
gm.predict(X); gm.predict_proba(X)     # hard and soft labels
gm.means_, gm.covariances_, gm.weights_
gm.score_samples(X)         # log density, low = anomaly
gm.bic(X); gm.aic(X)        # lower is better
```

- **Scale the features first.** All three methods use distances
- **n_init:** 10 is the k-means default. For GaussianMixture the default is 1, so set it yourself
- **random_state:** set it, then change it once to see whether the answer moves
- **DBSCAN on a map:** metric="haversine" with latitude and longitude in radians, eps in radians
- **covariance_type:** "full" is the default and the most flexible. "diag" is cheaper and good enough when features are many. "tied" shares one shape across components, "spherical" is k-means with soft labels
- **A field becomes a table:** mask the NaNs, index with the mask, transpose to samples by features. Keep the masked latitude and longitude to plot the result
- **Discrete colors for labels:** ListedColormap, or cmap="tab10" with vmin and vmax set to the label range